Unexpected Effects of Active Padding and Collator in Hugging Face Model Training

Vaniushkina Kseniia

Abstract

In this paper, I share my experience of training transformer-based models with the Hugging Face framework while experimenting with dynamic padding and bucketing. These techniques are often described as ways to speed up training and improve generalization. However, in my case, the results were different. Although the accuracy on the official test set increased, the model's predictions on sentences I created myself became noticeably worse. I also found that there is an opinion that using more padding tokens can prevent the model from overfitting to the training data and instead help it generalize better. This observation encouraged me to think about the hidden trade-offs in padding strategies and their role in real-world performance.

1 Introduction

When I first started working with transformer-based models using the Hugging Face framework, I came across several recommendations aimed at increasing training efficiency. Among them were two techniques: bucketing and dynamic padding. The documentation explained that these methods can speed up training and help models generalize better. The idea seemed simple: if unnecessary padding is reduced, the model wastes fewer resources on processing empty tokens and can instead focus more on real tokens.

I decided to apply these methods in my experiments. I trained models both with and without bucketing and dynamic padding, and then carefully compared their behavior. At first glance, the results looked promising: accuracy on the official test set did increase, which is usually interpreted as an improvement in quality.

However, when I tested these same models on sentences I created myself — sentences that were not part of the dataset but reflected realistic usage — I obtained unexpected results. The prediction quality decreased. Even though the evaluation reports showed progress, the model generalized worse in practice.

Reflecting on this outcome, I came across an interesting opinion shared by some practitioners: using more padding tokens may work as a form of regularization. In other words, extra padding might prevent the model from overfitting to the training distribution and instead encourage it to generalize better.

The purpose of this paper is to document my experience and discuss how padding strategies — often considered minor implementation details — can significantly influence not only benchmark scores but also the usability of models in real-world scenarios.

2 Background and Related Work

2.1 Bucketing

In natural language processing tasks, input sequences often vary greatly in length. To reduce the amount of redundant padding, the **bucketing** method was introduced. The idea is to group examples of similar length into the same batch. This reduces the number of padding tokens added, which decreases computational overhead and speeds up training.

Previous studies have highlighted that bucketing helps utilize GPU resources more effectively and reduces variability within batches (Sutskever et al., 2014; Britz et al., 2017). This is especially important when working with large corpora and long sentences, where redundant padding can occupy significant memory.

Overall, bucketing is considered a simple yet effective way to improve training efficiency without altering the underlying architecture of the model.

2.2 Dynamic Padding

Another popular technique is dynamic padding. Unlike static padding, where all sequences are extended to the same maximum length (e.g., 512 tokens), dynamic padding adjusts the length for each batch individually, based on the longest sequence within that batch.

In Hugging Face, this is implemented through data collators, such as DataCollatorWithPadding. The collator automatically trims or pads examples so that they match the batch's maximum length. This reduces the number of artificial tokens and can significantly accelerate training.

It is commonly believed that dynamic padding not only saves resources but also improves quality by exposing the model to cleaner data with fewer artificial tokens (Devlin et al., 2019; Hugging Face, 2023). Practitioners have reported that this strategy helps models generalize better and sometimes even boosts performance on test sets.

Together, bucketing and dynamic padding have long been regarded as universally beneficial. However, my experiments revealed that their effect can be more complex: a gain in benchmark accuracy does not always translate into better real-world performance.

3 Method

For the experiments, I used the NSMC dataset (Naver Sentiment Movie Corpus), a large Korean movie review dataset labeled for binary sentiment classification (positive/negative). It consists of 150,000 training examples and 50,000 test examples.

I trained two transformer architectures:
BERT (klue/bert-base),
RoBERTa (klue/roberta-base).

All models were trained with the same hyperparameters to ensure fair comparison:
learning_rate = 2e-5
batch_size = 16 (train) / 32 (eval) num_train_epochs = 1
weight_decay = 0.01 warmup_ratio = 0.1 max_grad_norm = 1.0
learning rate scheduler: linear (for BERT), cosine (for
RoBERTa)

The only difference lay in how padding was applied:

1. Model 1 (baseline) — static padding (padding="max_length", no collator, no bucketing).
2. Model 2 — dynamic padding + bucketing (DataCollatorWithPadding, group_by_length=True).
3. Model 3 — RoBERTa with the same dynamic padding and bucketing setup.

In addition to evaluating on the official test set, I also tested the models on a custom set of 10 hand-written sentences (5 positive, 5 negative). These sentences were never seen during training and were designed to mimic real-world usage.

4 Methods

Table 1: Performance on test and custom sets

Model	Padding Strategy	Epochs	Test Accuracy	Custom Accuracy
BERT (Baseline)	Fixed	1	0.9029	1.00
BERT	Dynamic + Bucketing	1	0.91	0.60
RoBERTa	Dynamic + Bucketing	1	0.9071	0.70

4.1 Model Predictions

Sentence (Korean)	BERT (Fixed)	BERT (Dyn+Bucket)	RoBERTa (Dyn+Bucket)
완전 최고예요. 다시 보고 싶습니다!	1 ✓	1 ✓	1 ✓
배우들 연기가 정말 훌륭했어요.	1 ✓	1 ✓	1 ✓
음악이 멋지고 여운이 오래 남아요.	1 ✓	0 ✗	0 ✗
연출이 섬세하고 몰입감이 뛰어났습니다.	1 ✓	0 ✗	1 ✓
기대 이상이었고 추천하고 싶어요.	1 ✓	1 ✓	1 ✓
스토리가 엉망이고 개연성이 전혀 없네요.	0 ✓	0 ✓	0 ✓
시간 낭비였습니다. 돈 아까워요.	0 ✓	1 ✗	1 ✗
지루하고 중간에 나가고 싶었어요.	0 ✓	1 ✗	1 ✗
연기가 어색하고 감동도 없었습니다.	0 ✓	0 ✓	0 ✓
별로였어요. 다시 보고 싶지 않습니다.	0 ✓	0 ✓	0 ✓

BERT (Fixed): 10/10 = 1.00

BERT (Dynamic + Bucketing): 6/10 = 0.60

RoBERTa (Dynamic + Bucketing): 7/10 = 0.70

4.2 Training graphs

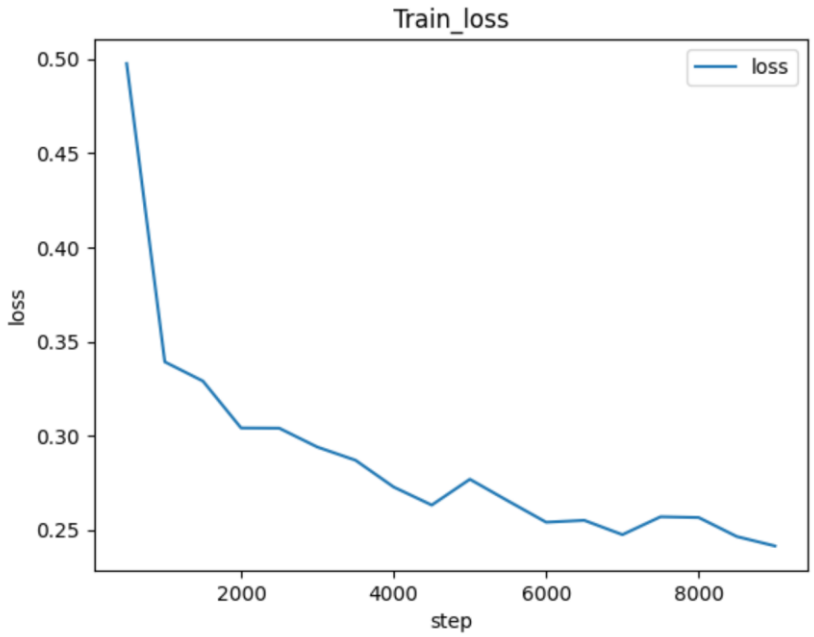


Figure 1. Training loss curve of BERT (fixed) model

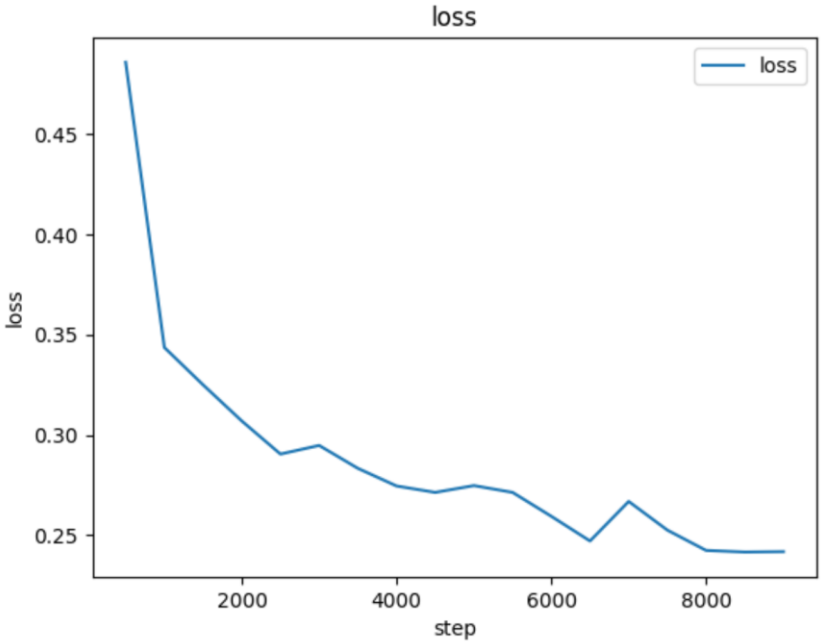


Figure 2. Training loss curve of BERT (Dynamic + Bucketing)

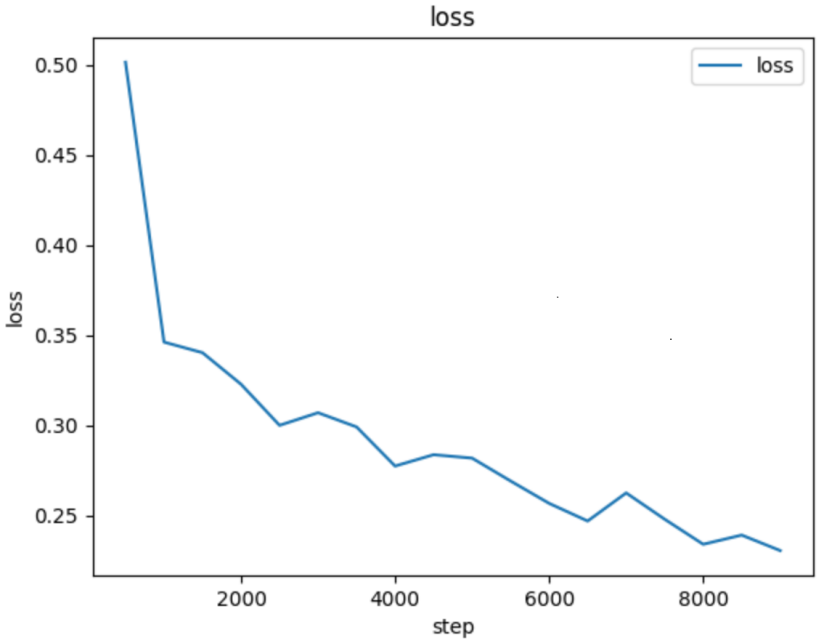


Figure 3. Training loss of RoBERTa (Dynamic + Bucketing)

5 Discussion and Conclusions

My experiments clearly show that what works well in documentation and benchmarks may not always translate into realworld benefits. Bucketing and dynamic padding indeed reduced redundant tokens and sped up training. On top of that, test set accuracy increased slightly, which on paper looked like progress.

Yet in practice, these optimizations reduced generalization ability. The static padding baseline, while computationally less efficient, predicted more reliably on unseen examples.

This led me to consider padding as a form of implicit regularization. By forcing the model to process more “empty” tokens, static padding may reduce overfitting to the training distribution and increase robustness.

The main conclusions are:

Benchmark metrics alone can be misleading;
Dynamic padding and bucketing are not guaranteed improvements but rather trade-offs; For applied projects, evaluation should always include realistic, user-generated examples.

Padding strategies, often considered minor implementation details, can have a surprisingly large impact on the real-world usability of models.

References

Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to Sequence Learning with Neural Networks. Advances in Neural Information Processing Systems (NeurIPS).

Britz, D., Goldie, A., Luong, T., & Le, Q. (2017). Massive Exploration of Neural Machine Translation Architectures. Proceedings of ACL.

Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. Proceedings of NAACL-HLT.
Hugging Face. (2023). Transformers Documentation.